실전반 1회차 — 사건 기록의 근거 추적 도구 만들기

요청을 받고 프로젝트 열기

앞에서 경보와 판단, 승인과 실행의 기록을 구별했습니다. 이번에는 정비 담당자가 작업지시를 보고 "왜 이 작업이 필요하다고 판단했는지 근거를 찾아 달라"고 요청한 상황입니다. 기록을 연결하고, 지금 자료로 확인할 수 있는 범위까지 보고하는 도구를 만들겠습니다. 실습 코드를 내려받아 압축을 풀고 VS Code에서 열겠습니다.

제가 아는 것은 요청의 목적입니다. 이 프로젝트에서 기록을 어떻게 읽고 결과를 어디에 남기는지는 아직 모릅니다. 먼저 코딩 도우미와 준비된 부분을 확인하겠습니다.

탐색 결과에서 작업 범위 정하기

도우미가 알려 준 위치를 열어 설명과 실제 자료를 맞춰 보겠습니다. 작업지시, 판단, 사건, 경보가 어떤 참조로 연결되는지 봅니다. 같은 설비 이름이 나온다는 이유만으로 같은 사건이라고 묶을 수는 없습니다. 기록 자체의 식별자와 다른 기록을 가리키는 값을 나누겠습니다.

이 입력은 제공된 실행 기록의 발췌입니다. 전체 사건 기록이 모두 들어 있는지는 별도로 확인해야 합니다. 작업지시가 등록된 사실과 정비가 끝난 사실도 구별하겠습니다. 오늘 도구가 판정할 것은 기록 사이의 직접 연결이며, 설비의 안전 상태나 정비 완료가 아닙니다.

파일 읽기와 저장 기능은 준비돼 있으니 활용하겠습니다. 연결을 비교하는 부분을 구현하고, 결과를 원문으로 되짚을 수 있는 보고서를 만드는 데 범위를 두겠습니다.

요구사항과 확인 기준 구체화하기

작업지시에서 판단으로, 판단에서 사건으로, 사건에서 경보로 이어지는 세 연결을 따로 판정하겠습니다. 비교할 값이 둘 다 있고 같으면 확인, 둘 다 있지만 다르면 불일치, 값이 없으면 미확인입니다. 기존 출력에서 불일치는 실패라는 상태로 표시하므로 그 이름과 뜻을 유지하겠습니다. 빈 문자열끼리 같다는 계산만으로 연결을 인정하지 않습니다.

원문에서 비교한 항목과 값, 그 위치를 판단 메모에 먼저 적겠습니다. 모든 연결이 확인되지 않으면 전체 경로도 확인됐다고 보고할 수 없습니다. 반대로 미확인이라는 결과만으로 실제 시스템의 연결이 잘못됐다고 단정해서도 안 됩니다.

계획에서 요청과 관계없는 기능이 들어갔는지 보겠습니다. 기록을 찾아 비교하고 설명하는 데 필요한 변경만 남깁니다. 이제 이 기준으로 구현을 요청하겠습니다.

구현 요청과 변경 검토

변경 내용을 업무 요구와 대조하겠습니다. 입력 자료를 바꾸지는 않았는지, 근거가 없는 연결을 추측하지 않는지, 기존 결과를 읽는 부분과 새 보고서가 호환되는지 확인합니다. 코드 줄을 차례로 강의하기보다 이 세 요구가 구현에 반영됐는지를 도우미의 변경 설명과 결과로 확인하겠습니다.

원문과 실제 결과 대조

원본 발췌를 읽고 적은 예상과 이제 나오는 결과를 비교하겠습니다. 각 연결의 판정뿐 아니라 비교한 두 값과 출처도 같은지 보겠습니다. 결과가 다르면 어떤 입력을 읽었고 어떤 조건으로 판정했는지부터 좁히겠습니다.

보고서의 근거 위치를 열어 보겠습니다. 한 연결의 확인 결과가 전체 사건의 확인으로 확대되지 않았는지, 필요한 다음 기록이 구체적으로 적혔는지 봅니다. 출처가 없는 내용을 그럴듯한 설명으로 채웠다면 보고서 생성 기준을 고치겠습니다.

다른 입력에서도 같은 기준 확인

제공된 가림 입력도 사용하겠습니다. 이것은 실제 장애 기록이 아니라 정보가 부족할 때의 판단을 확인하는 교육용 자료입니다. 정상 코드를 망가뜨리는 대신 입력의 차이가 판정과 보고서에 어떻게 반영되는지 보겠습니다.

불일치 입력을 정확히 불일치로 판정하면 그 검사는 통과입니다. 프로그램의 오류와 업무 자료의 불일치를 구별하겠습니다. 입력을 바꿔도 판정 규칙은 유지되고, 보고서에 이전 입력의 결과가 남아 있지 않아야 합니다.

저장하고 다시 근거 찾기

요구사항, 수정한 프로젝트, 두 입력의 결과와 보고서, 검사 기록을 저장하겠습니다. 작업 공간을 다시 열고 보고서의 연결 하나에서 원문으로 돌아가겠습니다. 누군가 이 작업지시의 이유를 물었을 때 확인된 근거와 추가로 필요한 기록을 구분해서 전달할 수 있어야 합니다.